HRMS — Phase 3

Time off: requests, approvals, balances and calendar — development and test report

Field	Value
Phase	Phase 3 — Time off (Phases 0–4 are the MVP)
Status	Backend verified on SQLite and PostgreSQL. Frontend typechecked and built; browser flow not verified.
Backend tests	258 passed, 2 skipped on SQLite · 260 passed on PostgreSQL 18
Frontend	tsc --noEmit exit 0 · vite build 479 kB JS / 144 kB gzip
Database	PostgreSQL 18 in development; btree_gist exclusion constraint applied
Date	2026-10-19
Handbook basis	docs/Handbook.pdf, Phase 3 (pages 9–10)

What this document is

A record of the Phase 3 work: the time-off models and rules, the services that move balances, the API, the React pages, the commands that were run with their real output, the defects found and fixed, and — explicitly — what has **not** been verified. Where something was not executed it is labelled as such rather than described as working.

Summary

- **Phase 3 is complete against the Handbook's checklist.** Leave types, yearly balances, requests and holidays exist; requests are approved, rejected or cancelled with correct balances; the calendar shows approved leave; the API is role-scoped; every state change is audited.
- **Tests were written first for timeoff/services.py**, as the Handbook asks. The first run of that file failed on a missing services module; the implementation then made all 47 non-Postgres service tests pass.
- **The overlap rule exists twice:** in the service, and as a PostgreSQL exclusion constraint over an inclusive daterange. The two Postgres-only tests are skipped on SQLite and were run against a real PostgreSQL 18 server.
- **The suite grew from 153 to 260 tests.** 258 pass and 2 skip on SQLite; all 260 pass on Postgres.
- **Eleven problems were found and fixed.** The most instructive was a PostgreSQL-only failure: *FOR UPDATE cannot be applied to the nullable side of an outer join*, triggered by locking a request through its employee's nullable team. SQLite never showed it.
- **A live HTTP run on Postgres proved the whole flow:** preview, request, overlap refusal, approval, balance deduction, calendar, cancellation and balance restoration, with audit actors.
- **Not verified:** a browser click-through, ruff (still blocked by the sandbox), and frontend lint/tests.

Phase 3 acceptance criteria (from the Handbook)

Handbook requirement	Evidence
A leave request goes end to end with correct balances	Live run on Postgres: 2-day request, approval 20 → 18, cancellation back to 20.

Handbook requirement	Evidence
Tests cover weekends, holidays, half days, overlaps, overruns and cancel; they pass	49 service tests and 58 API tests; 258 pass, 2 Postgres-only skip on SQLite.
The calendar shows approved leave	Calendar endpoint and page; live run showed the approved request in November, and cancellation removed it.
Watch out: balances drifting	Approve then cancel asserted to return to the starting value, in service and API tests.
Watch out: double approval deducting twice	Status check under a row lock; sequential double approve tested; database constraint covers the overlap race.

1. Scope and deliverables

Phase 3 goal (from the Handbook)

Leave management that replaces spreadsheets: request, approve, balance and calendar. The Handbook asks for LeaveType, LeaveBalance, LeaveRequest and Holiday models; working-day calculation, balance and overlap checks in code and in the database; approve, reject and cancel that keep balances correct inside a transaction; React pages for my time off, requests, approvals and a team calendar; an allocate_balances command; and an extended seed.

Delivered

Item	Where	Status
LeaveType, LeaveBalance, LeaveRequest, Holiday	apps/timeoff/models.py	Done, migrated
Unique balance per employee/type/year; non-negative balances	database constraints	Done
end_date >= start_date	database CHECK	Done
Overlap blocked for non-cancelled leave	Postgres exclusion constraint + service	Done, tested on both DBs
count_working_days (Mon–Fri, holidays, half day 0.5)	timeoff/services.py	Done, 8 tests
request / approve / reject / cancel / preview	timeoff/services.py	Done, 49 service tests
Locked balance and request rows in one transaction	select_for_update + atomic	Done
Leave types, balances, requests and calendar API	timeoff/views.py, urls.py	Done, 58 API tests
allocate_balances --year	timeoff/management/commands	Done, idempotent
Extended seed: types, holidays, balances, pending request	seed_users	Done, idempotent
Audit for leave types and requests	apps/audit/signals.py	Done, tested
My time off, request dialog, approvals, team calendar	frontend/src/pages/timeoff	Done, typechecked and built
README and docs/PERMISSIONS.md	docs	Done

Explicitly not in Phase 3

- No leave-type or holiday management screen. Leave types are writable through the API by HR/admin; holidays are managed in the Django admin or by the seed. The Handbook leaves that UI to Phase 4.
- No audit API or page (unchanged from Phase 2; rows are readable in the Django admin).
- No email notifications when leave is decided, no calendar export, no payroll integration, no multi-year requests.
- No new npm dependencies. The dialog, select and textarea are hand-built on the primitives already installed.

Files created and changed

```
backend/apps/timeoff/
  models.py  services.py  serializers.py  views.py  urls.py  admin.py  apps.py
  management/commands/allocate_balances.py
  migrations/{0001_initial}.py
backend/tests/
  test_timeoff_services.py  test_timeoff_api.py
backend/config/
  settings/base.py (app + Django floor)  settings/test_postgres.py
  urls.py (timeoff routes)
```

```
backend/apps/core/
  permissions.py (LeaveRequestPermission, IsHROrAdminOrReadOnly)
  scoping.py (scope_leave_requests, scope_leave_balances, scope_calendar_requests)
backend/apps/audit/signals.py (LeaveType, LeaveRequest tracked)
backend/apps/accounts/management/commands/seed_users.py (time off)
backend/requirements.txt (Django >= 5.1 for CheckConstraint.condition)
backend/build_phase03_pdf.py (this report's generator)
frontend/src/
  api/leave.ts  lib/date.ts
  components/leave/{LeaveStatusBadge.tsx,leaveColors.ts}
  components/ui/textarea.tsx
  pages/timeoff/{MyTimeOffPage,RequestLeaveDialog,ApprovalsPage,TeamCalendarPage}.tsx
  App.tsx  components/layout/navItems.ts  components/layout/Sidebar.tsx
  pages/DashboardPage.tsx
docs/PERMISSIONS.md  README.md  docs/Phase03.pdf
```

2. Data model and database rules

Model	Fields	Notes
LeaveType	name (unique), annual_days, is_paid, requires_approval, carry_over_max	Allowance per employee per year. requires_approval=False means a request is booked and deducted immediately.
LeaveBalance	employee, leave_type, year, allocated, used	Unique per employee+type+year; remaining = allocated − used. Read-only through the API.
LeaveRequest	employee, leave_type, start_date, end_date, half_day, days, status, reviewer, review_comment, created_at	days is computed when the request is made, so approving is a pure balance movement.
Holiday	date (unique), name	Never a working day; shown on the calendar.

Database constraints

- LeaveBalance is unique on (employee, leave_type, year), so there is exactly one pot of days per type per year.
- allocated and used are both CHECKed non-negative, so a bug cannot quietly push a balance below zero.
- LeaveRequest CHECKs end_date >= start_date.
- The overlap rule is a PostgreSQL exclusion constraint. It applies only to requests whose status is not cancelled, so a cancelled request frees its dates.

```
ALTER TABLE "timeoff_leaverequest" ADD CONSTRAINT "leave_request_no_overlap"
  EXCLUDE USING gist (
    "employee_id" WITH =,
    daterange(start_date, end_date, '[]') WITH &&
  )
  WHERE (status <> 'cancelled');
```

The range is inclusive at both ends ('**[]**'), because leave dates are inclusive: a request ending on the 5th and one starting on the 5th must conflict. PostgreSQL's default '**[)**' would miss that. The equality arm needs btree_gist, which the migration installs first.

Why the migration looks unusual

Django's schema editor cannot create an exclusion constraint on SQLite, and the test suite deliberately runs there. The generated CreateModel was therefore edited — before it was ever applied — so that the constraint is tracked in migration state but created by a vendor-guarded RunPython:

```
migrations.SeparateDatabaseAndState(
    state_operations=[migrations.AddConstraint(
        model_name='leaverequest', constraint=<exclusion constraint>,
    )],
    database_operations=[migrations.RunPython(
        create_overlap_constraint, drop_overlap_constraint,
    )],
)

create_overlap_constraint():
  if connection.vendor != 'postgresql': return
  CREATE EXTENSION IF NOT EXISTS btree_gist
  ALTER TABLE ... ADD CONSTRAINT ... (raw SQL above)
```

On SQLite the database operation is a no-op and the state still matches the models, so makemigrations --check reports no changes. On Postgres the extension and the constraint are real; psql shows the constraint after migrate, and the two Postgres-only tests prove it refuses overlapping rows.

3. Working days and the leave lifecycle

count_working_days

Inclusive of both dates, Monday to Friday, minus the company holidays in that range. A half day must be a single date and costs 0.5; on a weekend or holiday it costs 0 and the request is refused for having no working days. The same function powers the request validation and the live preview, so the browser never re-implements the calendar.

```
May 2026 (holiday on Wed 6 May, Founders Day)
  Mon 4  Tue 5  Wed 6*  Thu 7  Fri 8   ->  4 working days
  Mon 4 .. Sun 10                      ->  4 (weekend not counted)
  Tue 5, half_day=True                 ->  0.5
  Wed 6, half_day=True                 ->  0 (holiday)
  Sat 9 .. Sun 10                      ->  0  -> request refused
```

State transitions

```
pending --approve--> approved --cancel--> cancelled
pending --reject--> rejected
pending --cancel--> cancelled
approved / rejected / cancelled are terminal for review actions.

approve: status pending, reviewer may act, remaining >= days
         -> lock request + balance, used += days, status = approved
reject:  status pending, comment required -> status = rejected, no balance change
cancel:  pending or approved, owner or admin
         -> if approved: used -= days; status = cancelled
```

Balance safety

- Every state change runs inside transaction.atomic(). The request row and the balance row are locked with select_for_update() in a consistent order (request, then balance), so two reviewers cannot read the same remaining value and both pass.
- The balance is checked twice: when the request is made, and again at approval under the lock. Two pending requests can each fit the remaining balance; the second approval then fails with a clear message instead of overdrawing.
- Approving an already-approved request is refused by status, so a double click cannot deduct twice. The database exclusion constraint covers the other race: two requests for the same dates created at the same moment.
- Cancelling an approved request subtracts exactly the days that were added, so approve then cancel returns the balance to its starting value. Cancelling twice is refused.
- A type with requires_approval=False is created already approved and deducted in one save, which also keeps the audit trail to one row.

Authorisation rules

- A manager may review a request when the employee's manager is them, or when the employee is in a team they lead. Anything else is a 403 from the service; a request outside their scope is a 404 from the queryset before the service sees it.
- HR and admin may review anyone. HR has no employee record in some setups; the reviewer FK is then null, but the audit actor is still the signed-in HR user.
- Only the owner (or an admin) may cancel. HR deliberately cannot cancel someone else's leave.
- An account without an Employee record cannot request leave; the API says so plainly instead of failing on a missing relation.

Preview endpoint

The request dialog calls POST /leave-requests/preview/ on every date change. It returns the working-day count, the remaining balance, whether it is sufficient and whether the dates overlap. It writes nothing. This keeps holiday and half-day logic on the server and gives the dialog a real preview rather than an estimate.

4. API and permissions

Scoping

Role	Leave requests	Balances	Calendar
employee	own	own	own team's approved leave, plus own
manager	own, plus teams they lead	own	teams they lead, plus own
hr, admin	all	all	all

As elsewhere, out-of-scope rows are removed from the queryset, so a request outside your scope is a 404. The calendar only ever contains approved leave; pending and cancelled rows never appear on it.

Endpoints

Method	Path	Allowed	Notes
GET	/leave-types/	any signed-in	search; the request dialog needs them
POST / PATCH / PUT / DELETE	/leave-types/{id}/	hr, admin	deleting a type that has requests is a 400
GET	/leave-balances/	any signed-in, scoped	year filter; read-only
GET	/leave-requests/	any signed-in, scoped	status, leave_type, employee, year; paginated
POST	/leave-requests/	signed-in with an employee record	always for yourself
POST	/leave-requests/preview/	same	days, remaining, sufficient, overlap
POST	/leave-requests/{id}/approve/	manager (own team), hr, admin	deducts
POST	/leave-requests/{id}/reject/	manager (own team), hr, admin	comment required; no deduction
POST	/leave-requests/{id}/cancel/	owner or admin	restores an approved balance
GET	/leave-calendar/	any signed-in, scoped	year + month; approved leave and holidays
DELETE	/leave-requests/{id}/	nobody	405; cancel instead

Details that matter

- An employee field in the create body is ignored: the server sets the employee to the caller's record, so nobody can request leave on someone else's behalf.
- Balances are read-only. Only allocate_balances and the approval/cancellation services move them; there is no PATCH path that could desynchronise used from the requests.
- Leave types are writable by HR/admin, and deleting one that is referenced by a request is caught as a 400 rather than Django's protected-object 500.
- The calendar is one response for the month: approved entries plus the holidays in that range, so the page does not make a second call to place holidays.

- Filtering and pagination follow the project conventions: page-number pagination, unknown status and non-numeric ids are 400 field errors.

Example calendar response

```
GET /api/v1/leave-calendar/?year=2026&month=11
{
  "year": 2026, "month": 11,
  "results": [
    {"id": 2, "employee": 6, "employee_name": "Evan Employee",
     "leave_type": 1, "leave_type_name": "Annual",
     "start_date": "2026-11-02", "end_date": "2026-11-03",
     "half_day": false}
  ],
  "holidays": []
}
```

5. Audit trail

Phase 3 adds LeaveType and LeaveRequest to the models the audit signals track. The same rules apply: one save is one row, a no-op save writes nothing, and the actor is the signed-in user from the request middleware — null only for seed and other request-less writes.

```
request:  create  {"after": {..., "status": "pending", "days": "2.0"}}
approve:  update  {"changes": {"status": {"before": "pending", "after": "approved"},
                               "reviewer": {...}, "review_comment": {...}}}
cancel:   update  {"changes": {"status": {"before": "approved", "after": "cancelled"}}}
type:     create / update on timeoff.leavetype when HR edits one
```

- Auto-approved types produce one create row with status approved, not a create followed by an update, because the service saves the request once.
- Balance movements are not audited separately. The request row is the auditable event, and its status change records who did it; duplicating each balance write would double the trail for one decision.
- The reviewer FK and the audit actor are deliberately separate: HR and admin accounts can act without an employee record, so the actor is always the real user while reviewer may be null.
- Rows remain read-only in the Django admin and are only readable by the seeded superuser until an audit API/page is built.

6. Commands and seed data

allocate_balances

python manage.py allocate_balances --year 2026 creates one balance per active employee and leave type. It is idempotent: an existing row is left alone, so a second run does not overwrite an allocation HR adjusted. Carry-over is the previous year's remaining days, capped at the type's carry_over_max; it is added to the annual allowance. Inactive employees are skipped.

```
python manage.py allocate_balances --year 2026
Allocated 2026: 15 new balance(s) for 5 employee(s) and 3 leave type(s).

re-running: 0 new balance(s) (existing allocations kept)
```

Seed

Data	Value
Leave types	Annual 20 (carry 5), Sick 10, Unpaid 10 (is_paid=False)
Holidays	1 Jan, 1 May, 15 Aug, 25 Dec of the current year
Balances	5 employees × 3 types = 15
Requests	One pending Annual request for Esha: next Monday–Tuesday, 2 days

The pending request exists so the approvals page is not empty on a fresh demo. If the next Monday falls across a year boundary the seed skips it rather than failing. The seed remains idempotent: leave types are get_or_create by name, holidays by date, balances by the allocate command, and the demo request only when the employee has none.

> **A seed fix worth recording**
>
> The Unpaid type was first seeded with annual_days = 0, which made it impossible to request: a 0-day allowance fails the balance check. It now has 10 days. Leave types with a 0 allowance remain valid — they simply cannot be used until HR allocates days — but the demo should not contain a dead type.

7. Frontend

Page	Route	Behaviour
My time off (employee, manager)	/time-off	Balance cards per type with a used/allocated bar, a table of the caller's own requests (mine=true) with status badges, cancel for pending/approved, and an empty state when no balances are allocated.
My time off (HR, admin)	/time-off	A searchable directory of active employee names only. Clicking a name opens that person's page. It never lists leave on this screen.
Person time off	/time-off/employees/:id	The person's leave balance boxes, followed by their requests with status and requested date. HR/admin only; the API filters balances and requests by employee.
Request dialog	from /time-off	Leave type, date range, half-day toggle (single day only), and a live preview: working days, remaining balance, insufficient and overlap warnings. Submit is blocked when the preview says no.
Leave status (HR, admin)	/approvals	Every request in the company, newest first, with employee name, type, dates, requested date, status, reviewer and comment. Pending rows can be approved or rejected in place.
Approvals (manager)	/approvals	The pending queue for their team, with the same comment and decision controls. Status history stays hidden behind the API's scope for this role.
Team calendar	/team-calendar	Monday-first month grid with previous/next/today, leave chips coloured by type, holiday names, a legend, and a count instead of names at phone width.

How it is wired

- src/api/leave.ts holds every query and mutation: types, balances, requests, preview, calendar, create, approve/reject and cancel. Components never fetch.
- My time off asks for mine=true, so even a manager's team scope is narrowed to the caller. The HR/admin per-person page asks for employee= on both balances and requests.
- HR/admin see the unfiltered request list as a ledger; managers keep the status=pending queue. Both are ordered newest first by the API (created_at desc, id desc).
- The preview is a POST inside a TanStack Query keyed on the dates and type; it is disabled until the range is valid and does not retry on failure.
- After any mutation the ["leave"] namespace is invalidated, which refreshes balances, the request list, the calendar and the preview without hand-written cache edits.
- Date helpers in src/lib/date.ts avoid timezone drift: dates are built at local midnight and formatted from ISO strings, never through UTC parsing.
- No new dependencies: a native select, a hand-written modal (focus trap, Escape, scroll lock) and a small textarea cover the new UI.
- Route guards: /time-off/employees/:id and /employees are admin/hr; /approvals is manager/hr/admin; /time-off and /team-calendar are open to every role because the API scopes them.

> **Not verified in a browser**
>
> The React code typechecks and the production bundle builds, and every endpoint behind these pages was exercised over HTTP. But nobody has rendered the pages, opened the request dialog, typed a rejection comment, changed the calendar month or looked at the grid at 375px. This section describes implemented behaviour, not observed behaviour.

8. Verification performed

Every command below was run; the output shown is the real output.

8.1 Checks, migrations and seed (Postgres)

```
python manage.py check                          -> no issues (0 silenced)
python manage.py makemigrations --check --dry-run -> No changes detected
python manage.py migrate --noinput              -> timeoff.0001_initial applied
python manage.py seed_users                     -> 15 balances, 5 people, 3 types, 1 pending request

psql \d timeoff_leaverequest
  leave_request_no_overlap EXCLUDE USING gist (
    employee_id WITH =, daterange(start_date, end_date, '[]') WITH &&
  ) WHERE (status <> 'cancelled')
  leave_request_end_not_before_start CHECK (end_date >= start_date)
```

8.2 Test suites

```
pytest                                  -> 258 passed, 2 skipped (SQLite)
pytest --ds=config.settings.test_postgres -> 260 passed (PostgreSQL 18)

Phase 2 had 153 tests. Phase 3 adds 107:
  49 in tests/test_timeoff_services.py (2 are Postgres-only)
  58 in tests/test_timeoff_api.py
```

8.3 What the tests cover

Area	Examples
Working days	full week, weekend span, holiday excluded, Saturday alone, half day, half day on a holiday, end before start, half day over several dates.
Requesting	pending request with computed days, half day 0.5, cross-year refused, no-working-days refused, overrun refused, missing balance explained, overlap with pending refused, cancelled dates reusable, auto-approved type deducted immediately.
Approving	manager approves and balance drops, double approve refused, second approval overdraw refused and stays pending, manager outside team refused, employee refused, HR/admin allowed, rejected request cannot be approved.
Rejecting	comment required, no balance change, comment stored, approved request cannot be rejected.
Cancelling	pending no-op on balance, approved restores exactly, owner only, admin allowed, rejected refused, double cancel refused.
Preview	days and remaining, insufficient flag, overlap flag.
Allocation	15 rows, idempotent, carry-over capped at 5, existing allocation kept, inactive employee skipped.
API	role matrix on types/balances/requests, 404 for out-of-scope, spoofed employee ignored, mine and employee filters, filters and pagination, approve/reject/cancel status codes, calendar scope and holiday, preview, audit actors.
Database	two Postgres-only tests: an overlapping insert raises IntegrityError; a cancelled request does not block its dates.

8.4 Live HTTP run on Postgres

A throwaway client (removed afterwards) drove a real runserver on 127.0.0.1:8000 against PostgreSQL:

```
HR: 3 leave types, 15 balances, sees Esha's pending request
Evan: 3 own balances, preview 2 days and 0.5 half day, create (201, pending)
      overlapping request 400; approving own request 403
Manager: sees it pending, approves -> status approved, reviewer Meera Manager
Evan: Annual balance used 2, remaining 18; November calendar contains Evan
```

```
      cancel -> cancelled; second cancel 400; balance used 0, remaining 20
Seeded Esha request still pending; audit rows:
  create (employee2), update approve (manager), update cancel (employee2)
```

8.5 Frontend

```
tsc --noEmit   -> exit 0
npm run build  -> 1742 modules transformed in 7.98s
                  CSS 24.41 kB (gzip 5.66 kB)
                  JS 479.26 kB (gzip 144.30 kB)
```

8.6 Development database after the runs

```
leave types 3   balances 15   holidays 4   requests 2   audit rows 18
requests: Esha pending (seeded), Evan cancelled (from the live check)
```

9. Defects and errors solved

Eleven problems were found and fixed. The first was invisible on SQLite and only appeared when the same tests were pointed at PostgreSQL; the rest were caught by the tests, by reading the generated migration, or by running the flow.

9.1 FOR UPDATE through a nullable outer join (Postgres only)

Severity. High — approving, rejecting and cancelling all failed on PostgreSQL, the database this project targets.

Symptom. The Postgres test run failed with NotSupportedError: "FOR UPDATE cannot be applied to the nullable side of an outer join" on every review action. SQLite had passed the same tests minutes earlier.

Root cause. The locked query used select_for_update() together with select_related("employee__team"), and employee.team is nullable, so Django produced a LEFT OUTER JOIN. PostgreSQL refuses to lock rows on the nullable side of such a join.

Fix. The locked query now locks only the leave request row and reads its relations separately inside the transaction. Those relations are not being modified, so they do not need the lock.

Verified. The full suite passes on PostgreSQL (255 tests) and still passes on SQLite. The error no longer appears in the server log.

9.2 An exclusion constraint that SQLite cannot create

Severity. Medium — the test database would fail to build, or the constraint would be missing on Postgres.

Symptom. The generated migration put the PostgreSQL exclusion constraint inside CreateModel, so applying migrations on SQLite failed.

Root cause. The constraint is a Postgres-only feature (GiST + btree_gist), but migration state is database-independent.

Fix. The constraint was moved out of CreateModel into SeparateDatabaseAndState: a state AddConstraint that keeps makemigrations in sync, and a RunPython that runs only when the vendor is postgresql, creating btree_gist and the constraint with explicit SQL.

Verified. makemigrations --check reports no changes; SQLite test databases build; psql shows the constraint on the dev database; both Postgres-only tests pass.

9.3 Two pending requests could overdraw at approval

Severity. High — the balance could go negative, which the whole feature exists to prevent.

Symptom. Each request was checked against the remaining balance when created, but a pending request does not reserve days. Two 5-day requests against 6 remaining both passed creation.

Root cause. The balance was only checked at request time, not again under the lock at approval.

Fix. Approval re-reads the balance with select_for_update() and refuses if the days no longer fit, with the remaining and requested counts in the message. A test approves the first and asserts the second stays pending with used unchanged.

Verified. test_an_approval_that_would_overdraw_is_refused passes on SQLite and Postgres.

9.4 Double approval deducting twice

Severity. High — the Handbook calls this out explicitly.

Symptom. A second approve request could have repeated the deduction.

Root cause. No status guard was assumed by the client; a double click or a retried request must be harmless.

Fix. The request row is locked, the status must still be pending, and only then is the balance locked and changed. A second call fails with “already approved” before touching the balance.

Verified. test_approving_twice_does_not_deduct_twice and the API equivalent pass; the Postgres exclusion constraint covers the same-dates race independently.

9.5 Cancellation had to return exactly what approval took

Severity. High — drifting balances were the Handbook's first warning.

Symptom. Cancelling an approved request needed to add the days back, but only if the request was actually approved, and only once.

Root cause. Pending requests never deducted, so cancelling them must not add; a second cancel must not add again.

Fix. The service checks the status, locks the balance only for approved requests, subtracts the stored days, and sets the status to cancelled. The API and service tests assert used returns to 0 exactly.

Verified. test_cancelling_an_approved_request_returns_the_balance passes; the live run showed 18 → 20 after cancellation.

9.6 Auto-approved types could write two audit rows

Severity. Medium — the “one action, one row” rule would have looked broken.

Symptom. A type with requires_approval=False is booked immediately, which is conceptually a create plus an approve.

Root cause. Calling approve_leave from request_leave would save the request twice: create, then update.

Fix. The service computes the final status before its single create, deducts the balance in the same transaction, and saves once. The audit signal therefore writes one create row whose changes already show status approved.

Verified. test_a_type_without_approval_is_approved_immediately asserts the balance, and the audit row is a single create.

9.7 HR reviewers without an employee record

Severity. Medium — HR could be blocked from doing its job, or the audit could lose the actor.

Symptom. The reviewer field points at an Employee, but an HR or admin account may not have one (the tests create such accounts).

Root cause. Setting request.reviewer = request.user is impossible, and skipping the attribution would make the decision anonymous.

Fix. The service resolves the actor's Employee record when it exists and stores null otherwise; the audit actor is always the signed-in user from the middleware, so the decision remains attributable.

Verified. test_hr_can_approve_anyone asserts reviewer None and the audit test asserts the actor is the HR user.

9.8 Deleting a leave type with requests

Severity. Medium — an HR action would have produced a 500.

Symptom. DELETE /leave-types/{id}/ raised Django's ProtectedError when any request referenced the type, because the FK is PROTECT.

Root cause. The viewset had no handling for protected objects; DRF only knows HTTP exceptions.

Fix. perform_destroy catches ProtectedError and raises a 400 explaining that the type has requests. Balances still cascade; requests do not.

Verified. test_a_type_with_requests_cannot_be_deleted expects the 400.

9.9 A serializer regression made a field required

Severity. Medium — creating a leave type from the API failed for no obvious reason.

Symptom. The API test for creating and deleting a type got 400 "This field is required" for carry_over_max. The model has a default, but the serializer declared the decimal fields explicitly, and explicit fields are required by default.

Root cause. Overriding a ModelSerializer field drops the model-derived required=False.

Fix. Both decimal fields are declared with required=False and a Decimal default; partial updates are unaffected.

Verified. test_hr_can_create_a_type and the full API suite pass.

9.10 The seeded Unpaid type was unusable

Severity. Low for code, high for the demo.

Symptom. Unpaid had annual_days = 0, so any request against it failed the balance check with "0 remaining".

Root cause. A balance is still required per type, and the seed allocated the model default of zero.

Fix. The seed gives Unpaid 10 days and is_paid=False, which is a plausible policy and makes the type usable in the demo.

Verified. The dev database was updated to match; the type now shows 10 allocated in the balances API.

9.11 My time off showed other people's leave

Severity. High — a page titled My time off listed HR's whole company and a manager's whole team, without even showing whose row it was.

Symptom. Reviewing the running app, HR/admin saw every employee's requests on My time off, and the table had no name column, so the rows were indistinguishable. The page name also promised the caller's own leave, so the behaviour was both confusing and a privacy problem.

Root cause. The page requested /leave-requests/?page_size=100 without narrowing. The API correctly scopes HR/admin to all and managers to their team, so the page inherited that breadth and presented it as "mine".

Fix. A mine=true filter narrows the list to the caller's employee record (empty when there is none), and My time off now uses it. HR/admin instead get a searchable directory of names; a person's page uses employee= on balances and requests. The Approvals page became the explicit all-request ledger for HR/admin, newest first, with names, statuses, reviewers and decisions.

Verified. Five new API tests cover mine and the employee filter, including that an employee cannot widen scope with employee=. The full suite is 258 passed / 2 skipped on SQLite and 260 on Postgres; the dev database returned HR mine=0, manager mine=1 and employee mine=1.

> **Toolchain, again**
>
> npm run build still needs wider-than-sandbox access because esbuild starts a helper process with piped output, which the Windows sandbox blocks (spawn EPERM). It succeeded once run that way. ruff remains declared in requirements.txt but could not be installed in this sandbox, so no lint run happened. The empty .tmp/pip-* directories from the blocked installer are still present and gitignored.

10. What is not verified

Item	Status
React pages in a browser	NOT DONE. The bundle builds and the APIs were exercised over HTTP, but no page has been rendered or clicked.
375px layout, month grid, dialogs	NOT DONE. Code-only; responsive classes and the calendar grid are unobserved.
True parallel approvals	PARTIAL. Row locks and the database constraint are in place and sequential double approval is tested; a two-connection race test was not written.
ruff lint	NOT RUN. Not installed; installation is blocked by the sandbox.
Frontend lint and tests	NOT AVAILABLE. ESLint and Vitest were never set up; typecheck and build are the checks used.
Django admin forms	PARTIAL. Leave types and holidays are registered; editing through the admin was not exercised.
Leave-type / holiday UI	NOT BUILT. Leave types are API-only; holidays are admin/seed only. Phase 4.
Audit API or page	NOT BUILT. Rows are only readable in the Django admin.
Leave decision emails	NOT BUILT. The Handbook does not require them in Phase 3.
PostgreSQL earlier versions	NOT TESTED. Only PostgreSQL 18 was used; the SQL is standard for 12+.

Known limitations to carry forward

- A request cannot span two calendar years. That is deliberate: balances are per year, and splitting a request would need a policy decision and two balance movements.
- Carry-over is added to the new year but is not subtracted from the old year's balance, so the old row still reads as unused. It is capped at carry_over_max.
- allocate_balances never overwrites an existing allocation. Changing a type's annual_days does not retroactively change balances that were already created; HR would adjust them deliberately (there is no UI for that yet).
- The calendar shows approved leave only and includes teammates' names and leave types; that is the intended team-calendar behaviour, not a leak.
- The request dialog's preview is advisory; the server re-validates everything on submit.
- The frontend shows leave in the current year only for balances, while the request list shows the most recent 100 requests across years.

11. Reproduce the phase

```
cd D:\HRMS\backend
.venv\Scripts\python.exe manage.py check
.venv\Scripts\python.exe manage.py migrate
.venv\Scripts\python.exe manage.py seed_users
.venv\Scripts\python.exe -m pytest -q

# the Postgres-only constraint tests
$env:DATABASE_URL="postgres://postgres:postgres@localhost:5432/hrms"
.venv\Scripts\python.exe -m pytest -q --ds=config.settings.test_postgres

.venv\Scripts\python.exe manage.py runserver

# second terminal
cd D:\HRMS\frontend
npm run typecheck
npm run dev
```

Demo script (5 minutes)

- Sign in at http://localhost:5173 as employee@hrms.test / DemoPass123! and open My time off.
- Request Annual leave for two working days; the dialog shows the working-day count and remaining balance.
- Sign in as manager@hrms.test, open Approvals, add a comment and approve it.
- Sign in as hr@hrms.test: My time off now lists employee names only. Open Esha and see her balance boxes and requests.
- Still as HR, open Approvals (titled Leave status): every request, newest first, with names, statuses and decisions.
- Back as the employee, the balance has dropped by those days; open Team calendar and find the leave.
- Cancel the request from My time off; the balance returns to its starting value.
- Try to request overlapping dates: the dialog warns before submit, and the API refuses it anyway.
- As admin@hrms.test open /admin/ and browse Audit log to see the create, approve and cancel rows.

> **Bottom line**
>
> Phase 3 is complete and verified on both databases: 258 tests pass on SQLite (2 Postgres-only skip) and 260 pass on PostgreSQL, where the exclusion constraint is real. The working-day, balance and overlap rules live in one service file; approvals and cancellations are transactional and audited; the React pages compile and the APIs behind them were exercised live. The honest gaps remain the browser, ruff and frontend lint.